Lab 4 Report

1. What does your program do?

The program generates PWM waves within an infinite loop to increase and decrease perceived brightness of an LED whether a button is pressed or not. The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost.

2. What variables/registers does your program use?

The program uses the **DDRB** register to set the LED as an output and the button as an input.

The **PORTB** register to enable the pull-up property of the button as an input.

The **TCCR0A** register to set Timer 0 to CTC mode.

The **TCCR0B** register to set the prescaler to x1024 and start Timer 0, where with a clock frequency of 16MHz corresponds to a period of 16384 microseconds, or 16.384 milliseconds.

The **OCR0A** register acts as Timer 0's max value with which the counter would reset to 0 upon reaching, effectively the fraction of the pre-scaled period that Timer 0 would elapse before resetting.

The **OCR0B** register acts as Timer 0's duty cycle, having a value between 0 (0%) and OCR0A (100%) to determine for how long of the period the program should output High for.

The **TIFR0** register reads OCR0A and OCR0B's overflow flags and allows the program to wait for a period of time before switching from output high to low.

The **PINB** register to read whether the on-board button is pressed or not.

3. What is the main algorithm/logic of your program?

The main program calls the **init** function to initialize the DDRB register to set the on-board LED as an output and the on-board button as an input, with its pull-up property enabled to have a default value of "1" when not pressed.

Then it calls the **timer_init** function to set Timer 0's mode to CTC which will cause the clock to reset upon reaching the value stored in OCR0A. We set OCR0A to 155 (0x9B) so that 156 time steps occur before the clock resets, which when using a pre-scale value of 1024 means about 9984 microseconds pass which is as close to the desired 10 milliseconds that can be achieved given an 8 bit register. TCCR0B is used to set the prescaler value to 1024 and start the timer.

The **generate_PWM** function uses OCR0B acting as any duty cycle between 0% and 100%, but not including these cases. The LED is turned on until the OCF0B flag is raised, after which the flag is reset and the LED is turned off. The program waits until the OCF0A flag is raised, after which the flag is reset. OCF0A acts as the period for the PWM waveform while

OCF0B acts as the duty cycle. This function can't handle 0% and 100% duty cycle as there is an extremely small window where the LED is on before being turned off despite a duty cycle of 0% (very noticeable), and vice versa for a duty cycle of 100% (much less noticeable). I opted to handle both cases in the main program instead, by forcing the LED constantly on or off for 100% and 0% respectively as a part of overflow prevention.

The program enters the infinite while loop, where it will check whether the on-board button is pressed or not. If so, then the program prevents overflow of the OCR0B register by checking whether the OCR0B register is less than 100% (value of OCR0A) where it runs **generate_PWM** with OCR0B as the duty cycle and increases the register by one. If the button is still being pressed and OCR0B is not less than OCR0A (OCR0B is greater than or equal to OCR0A), then the overflow prevention takes place and constantly keeps the LED on to achieve a 100% duty cycle. If the button is not being pressed or has been released, then, as long as OCR0B is above zero, **generate_PWM** is run and the register is decreased by one. If the button is still not pressed and OCR0B is not above zero (since OCR0B is unsigned, is zero), overflow prevention takes place where the while loop continues without turning the LED on at all which achieves a 0% duty cycle.

4. What external hardware connections did you make?

There are no external hardware connections made since the on-board button and LED are used for input and output.

Appendix (Code)

```
#define F_CPU 16000000UL // 16MHz clock speed

#include <avr/io.h>


void init(void) { // initialize i/o
	DDRB |= (1 << DDRB5); // output to LED

	DDRB &= ~(1 << DDRB7); // set on board button to input
	PORTB |= (1 << PORTB7); // pull-up resistor for on-board button, default value (OFF) is
1
}

void timer_init(void) { // initialize timer settings
```

```
	// FCPU = 16MHz, want 10ms
	// ps = 1, NM -> 16us
	// ps = 1024, NM -> 16384us = 16.384ms

	// Set to CTC Mode
	TCCR0A |= (1 << WGM01);

	// 256 steps * (10000us/16384us) = 156.25 steps - 1 = 155.25 (round down) = 155
	// ~10ms timer, 0x9B == 155
	OCR0A = 0x9B;

	// start at 0% duty cycle
	OCR0B = 0x00;

	// Pre-scaler set to 1024, STARTS the timer
	TCCR0B |= (1 << CS02) | (1 << CS00);

}

void generate_PWM(void) { // generates a PWM waveform with variable duty cycle
	PORTB |= (1 << PORTB5); // LED on, HIGH
	while ( (TIFR0 & (1 << OCF0B)) == 0 ) {} // busy while until OCF0B flag == 1 in TIFR0
register
	TIFR0 |= (1 << OCF0B); // reset OCR0B overflow

	PORTB &= ~(1 << PORTB5); // LED off, LOW
	while ( (TIFR0 & (1 << OCF0A)) == 0 ) {} // busy while until OCF0A flag == 1 in TIFR0
register
	TIFR0 |= (1 << OCF0A); // reset OCR0A overflow flag
}

int main(void) {
	init();    // set inputs and outputs
	timer_init(); // timer settings

   while (1) {
		if ( !(PINB & (1 << PINB7)) ) { // button is pressed
			if (OCR0B < OCR0A) {
				generate_PWM();
				++OCR0B; // increase duty cycle
			}
			else PORTB |= (1 << PORTB5); // don't turn LED off if OCR0B is OCR0A
		}
		else { // button is not pressed/released
```

```
                        if (OCR0B > 0) {
                                generate_PWM();
                                --OCR0B; // decrease duty cycle
                        }
                        else continue; // don't turn LED on at all if OCR0B is 0
                }
        }

}
```